

锂离子电池快充策略对寿命衰减的影响建模与优化

摘 要

提高充电倍率可缩短补能时间,但过高电流会加速容量衰减。本文基于 MIT-Stanford 公开的 49 块 A123 18650 型磷酸铁锂电池循环老化数据,围绕两阶段快充策略 $C_1-Q_1-C_2$, 研究快充参数对寿命衰减的影响,并讨论寿命预测与策略优化问题。

针对问题一,本文对循环数据进行逐电池、逐字段的 IQR 异常检测与前后邻域均值填补,共替换 559 个异常单元格;提取 9 种快充策略,并以第 200 次循环的 SOH 作为观测窗口内的寿命代理指标(数据尚未覆盖 $\text{SOH} = 80\%$ 的真实寿命终止)。在 40 块满 200 圈电池中,SOH 均值为 0.9914,按 25%/75% 分位数划分得到长寿命 10 块、中等寿命 20 块、短寿命 10 块。策略层面,寿命最长为 $3.6\text{C}(80\%) - 3.6\text{C}$,寿命最短为 $3.7\text{C}(31\%) - 5.9\text{C}$ (NEWSTRUCTURE)。同名参数但带/不带 NEWSTRUCTURE 标签的策略寿命差异明显,提示批次或实验结构不可忽略。

针对问题二,本文以 SOH_{200} 和 200 圈衰减量为寿命代理指标,在同一实验结构的 34 块完整样本上完成策略间 Kruskal-Wallis 检验、参数关联/稳健回归诊断与 SOH 纵向混合效应分析。策略间总体差异显著 ($H = 12.3346, p = 0.0305$),但 Holm 校正后未识别出显著的单一策略对;参数共线性较强,故结果仅作样本内关联解释。

针对问题三,拟利用截至第 150 次循环的早期数据,提取健康特征并建立 SOH 衰减预测模型,预报第 151–200 次循环 SOH,并外推至 $\text{SOH} = 80\%$ 的循环寿命;同时比较不同早期窗口长度及策略信息的作用。(本节定量结果待补全)

针对问题四,拟建立充电时间模型与寿命衰减模型,在已有实验策略及其合理邻域内进行充电时间–寿命的多目标权衡,给出推荐策略并讨论外推风险。(本节定量结果待补全)

关键词: 锂离子电池; 快充策略; SOH; 循环寿命; IQR; 寿命预测; 多目标优化

1 问题重述

随着新能源汽车与储能系统对补能效率的要求提高，锂离子电池快充成为缩短充电时间的主要手段。然而较高充电电流会增强极化、加剧热效应并加速副反应，从而加快容量衰减、缩短循环寿命 [1–3]。电池健康状态定义为

$$\text{SOH} = \frac{Q_{\text{current}}}{Q_{\text{initial}}}, \quad (1)$$

本题以 $\text{SOH} = 80\%$ 作为寿命终止 (EOL) 判定阈值。

本题充电过程采用两阶段快充：以倍率 C_1 从 0% SOC 充至切换点 Q_1 ，再以倍率 C_2 从 Q_1 充至 80% SOC，随后以 1C 恒流–恒压 (CC-CV) 补至满电，至电流降至 $C/50$ 结束。各电池放电条件一致，主要差异在充电策略。数据来自 MIT–Stanford 公开循环老化实验 [1, 4]，包含 49 块额定容量 1.1 Ah 的 A123 18650 磷酸铁锂电池。

需要完成的研究任务为：

- 问题 1.** 整理数据，提取策略、充电时间、SOH 变化与循环寿命信息，分析寿命分布并解释典型长/短寿命策略差异。
- 问题 2.** 分析策略参数 C_1, Q_1, C_2 对循环寿命的影响及其统计显著性与可能机制。
- 问题 3.** 基于截至第 150 次循环的数据预测第 151–200 次循环 SOH，并外推 EOL 循环寿命。
- 问题 4.** 兼顾充电时间与寿命衰减，在实验策略及其合理邻域内优化快充策略。

2 问题分析与总体思路

问题一属于探索性数据分析：在尚未达到真实 EOL 的观测窗口内，需要定义可比较的寿命代理指标，并完成清洗、可视化与典型策略提取。问题二强调参数非正交，不宜按完全析因实验解释因果关系。问题三是早期循环预测与外推。问题四是充电时间与衰减率的多目标决策。四问递进关系为：数据整理 → 影响分析 → 寿命预测 → 策略优化。

待完成 / 框架占位

问题三至问题四的模型推导、数值表与最终推荐策略尚未写入仓库，下文相应小节仅给出可直接填充的结构、符号与拟采用方法，待后续结果补入后删除本框并替换为正式论述。

3 符号说明与基本假设

3.1 主要符号

表 1 主要符号说明

符号	含义
SOH	电池健康状态, 见式 (1)
C_1, C_2	第一、二阶段充电倍率 (C-rate)
Q_1	阶段切换 SOC (%)
N	循环次数
N_{EOL}	SOH = 80% 对应的循环寿命
SOH_{200}	第 200 次循环的 SOH (寿命代理)
T_{charge}	平均充电时间 (min)

3.2 基本假设

1. 各电池放电协议一致, 寿命差异主要由充电策略及电池个体差异引起。
2. 在 200 次循环窗口内未达到 $\text{SOH} = 80\%$, 故问题一以 SOH_{200} 作为寿命代理指标, 并在文中明确其与真实循环寿命的区别。
3. 带 NEWSTRUCTURE 后缀的策略与同名无后缀策略视为不同实验结构/批次标签, 分析时不合并。
4. IQR 筛出的离群点视为统计异常而非已证实的传感器故障, 清洗仅用于本题指定分析。
5. 问题四若扩展连续参数, 取值限制在已有实验数据的凸包或合理邻域内, 避免无约束外推。

4 问题一：数据整理与快充策略对寿命影响的初步分析

4.1 数据来源与结构

本题使用赛题提供的 MIT-Stanford 锂离子电池循环老化数据 [1,4]。battery_summary.csv 记录 49 块电池的策略参数 C_1, Q_1, C_2 、初始容量、平均充电时间、平均内阻、平均温度及问题三测试标记 prediction_test; cycle_train.csv 记录逐循环的容量、SOH、平滑 SOH、充电时间、内阻与平均温度, 共 9350 行。非测试电池记录至第 200 次循环, 9 块测试电池仅至第 150 次循环。输入质量检查表明: battery_id+cycle 无重复; 循环表无缺失; 汇总表有 3 个缺失单元格, 其中 C_1 的结构性缺失予以保留, 不参与循环数据清洗。最大循环数分布为 $\{150:9, 200:40\}$ 。

4.2 寿命指标定义

题设循环寿命为 SOH 降至 80% 时的循环次数 N_{EOL} 。本数据集在 200 次循环内 SOH_{200} 范围为 0.9497 ~ 1.0002，均值 0.9914，尚未覆盖真实寿命终止。因此问题一采用

$$LifeProxy_i = SOH_i(N = 200) \quad (2)$$

作为观测窗口内的使用寿命代理：该值越高，表示前 200 圈健康保持越好。后文箱线图、分布图与策略标签均基于该代理指标。若需与问题三衔接，可进一步计算衰减率

$$FadeRate_i = \frac{SOH_i(1) - SOH_i(N)}{N - 1}, \quad (3)$$

并在问题二、三中统一使用。[1,5]

4.3 IQR 清洗

对每块电池、每个测量字段 (capacity、SOH、SOH_smooth、chargetime、IR、Tavg) 分别计算下、上四分位数 Q_1, Q_3 及四分位距 $IQR = Q_3 - Q_1$ 。将

$$x \notin [Q_1 - 1.5 IQR, Q_3 + 1.5 IQR]$$

的观测标为异常，并用同一电池、同一字段中距离最近的前后正常值均值替换；若仅有单侧邻域则用该侧值。共替换 559 个单元格，分字段数量为：capacity 36、SOH 36、SOH_smooth 17、IR 142、Tavg 142、chargetime 186。原始 data/ 文件未改写，清洗结果保存为 output/A/cycle_train_cleaned.csv。

4.4 策略提取与 200 圈样本

由汇总表提取 49 块电池的 battery_id-policy 对照，得到 9 种策略（表 2 给出示意，完整表见支撑材料 charge_policy.csv）。寿命统计分析仅纳入最大循环数恰为 200 的 40 块电池，排除 9 块问题三测试电池，以免 150 圈终点与 200 圈终点混比。

表 2 部分电池快充策略提取结果（完整 49 行见支撑材料）

battery_id	policy
1	3_6C-80PER_3_6C
4	80PER_3_6C
7	4_8C_80PER_4_8C
10	5C_67PER_4C_NEWSTRUCTURE
16	3_7C_31PER_5_9C_NEWSTRUCTURE
49	4_8C_80PER_4_8C_NEWSTRUCTURE

4.5 SOH 变化曲线

以 1 号电池为例绘制清洗后 SOH 与平滑 SOH 随循环次数的变化，见图 1。早期循环中 SOH 可略高于 1（容量活化/测量波动），随后缓慢下降。该图用于展示单电池衰减形态，策略间对比见后文分布图与箱线图。

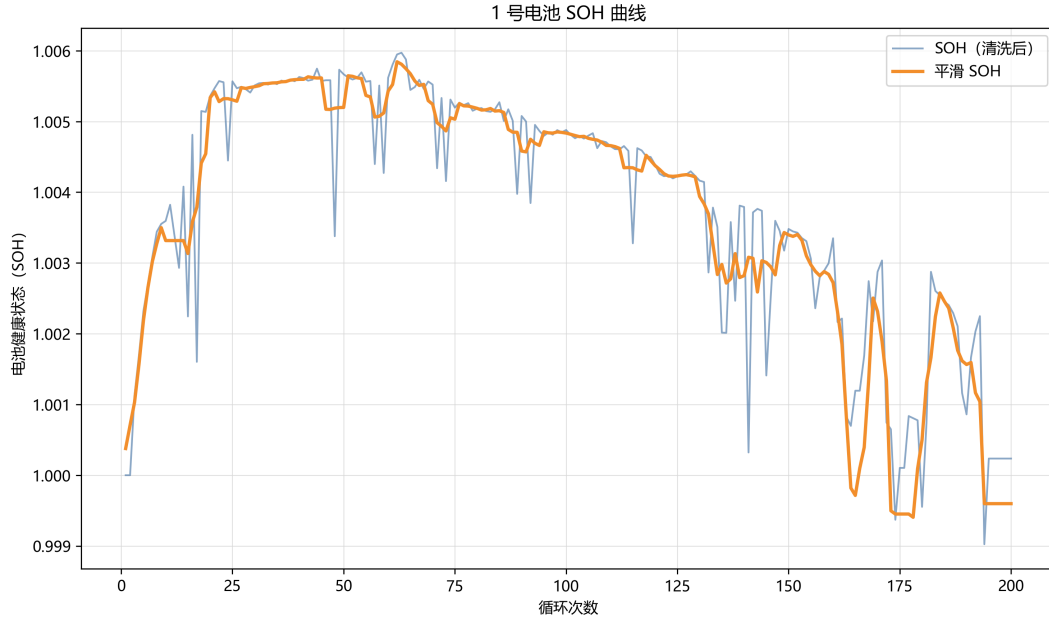


图 1 1 号电池 SOH 曲线（清洗后原始 SOH 与平滑 SOH）

4.6 寿命分布统计分析

对 40 块电池的 SOH_{200} 绘制直方图。取样本 75% 分位数

$$\text{critical_SOH}_{\text{high}} = 0.996103$$

为长寿命分界（红色虚线），25% 分位数

$$\text{critical_SOH}_{\text{low}} = 0.993433$$

为短寿命分界（蓝色虚线）。据此分类：长寿命 10 块、中等寿命 20 块、短寿命 10 块。分布见图 2。

SOH_{200} 与平均充电时间的散点图见图 3，按策略着色。可见充电时间较短的策略并不一律对应更低 SOH，时间-寿命关系并非简单单调，为问题四的多目标权衡提供现象基础。

不同策略下 SOH_{200} 的箱线图见图 4，并叠加上述两条分界线。各策略汇总统计见表 3。

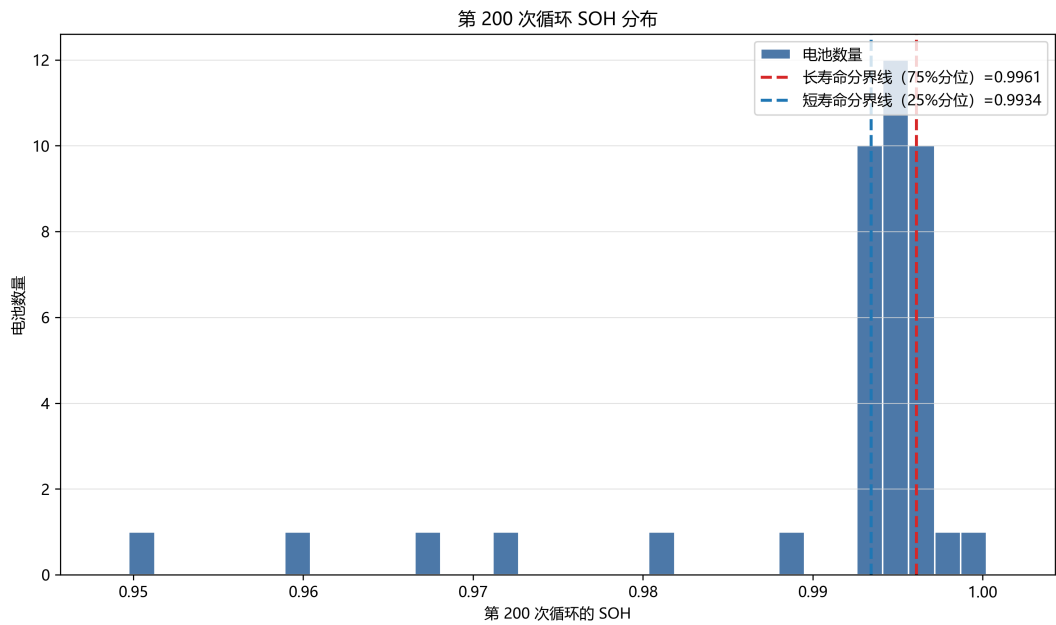


图 2 第 200 次循环 SOH 分布及长/短寿命分位数分界线

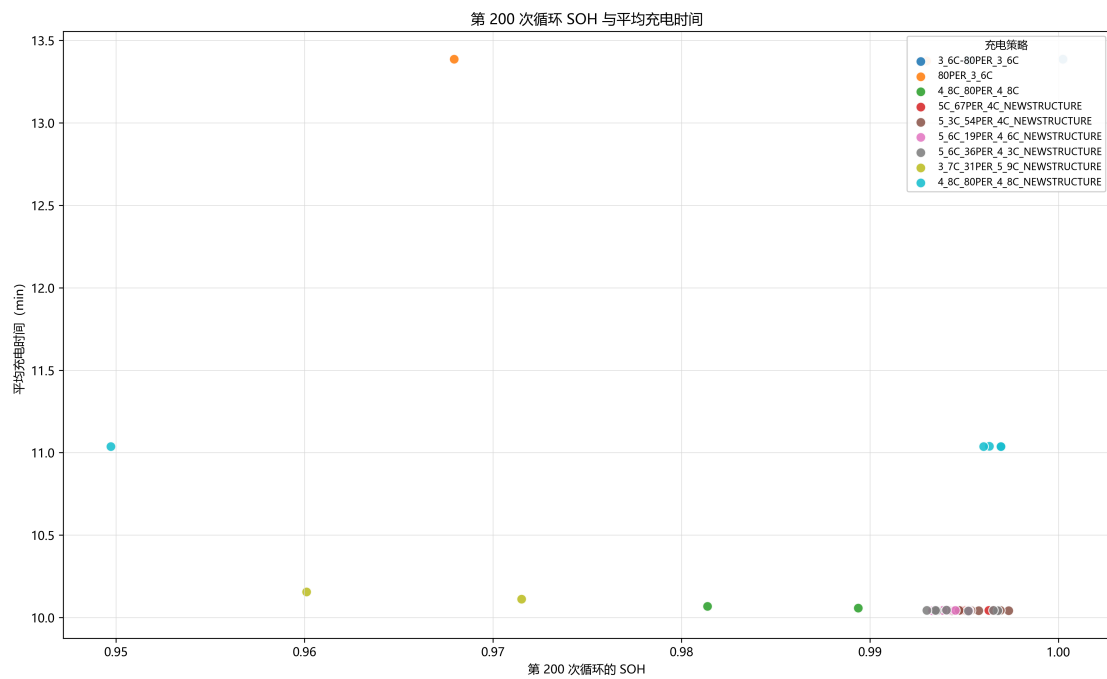


图 3 第 200 次循环 SOH 与平均充电时间散点图（按策略着色）

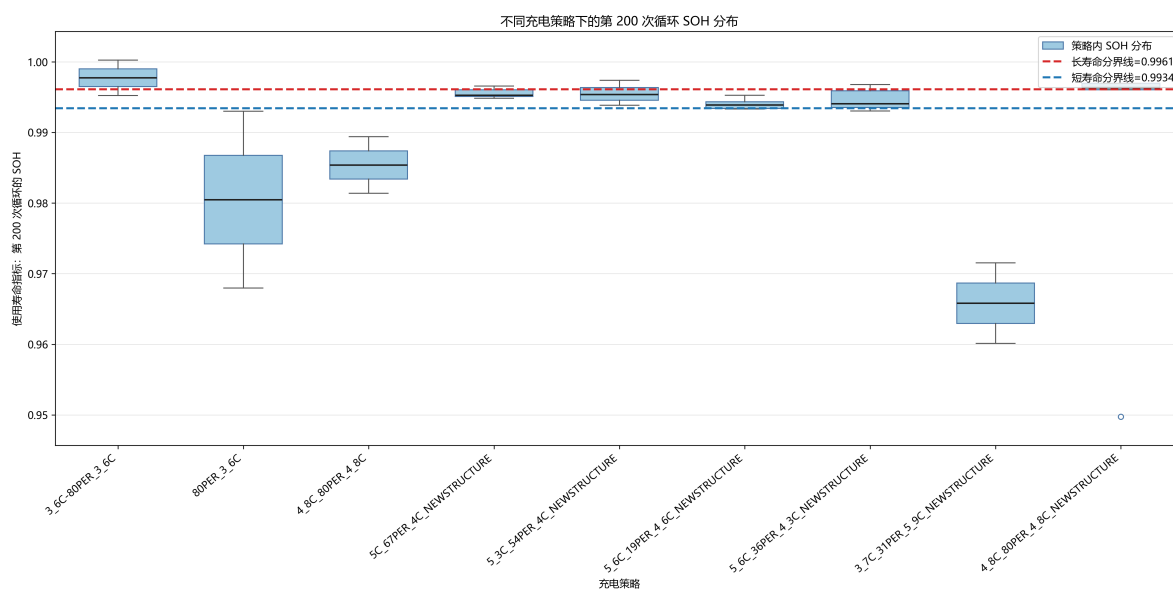


图 4 不同充电策略下第 200 次循环 SOH 箱线图

表 3 各充电策略第 200 次循环 SOH 与充电时间汇总（仅 200 圈样本）

策略	n	SOH 中位数	SOH 范围	充电时间范围/min	标签
3_6C-80PER_3_6C	2	0.9977	0.9952-1.0002	13.378-13.386	长寿命
4_8C_80PER_4_8C_NEWSTRUCTURE	5	0.9963	0.9497-0.9970	11.038-11.038	长寿命
5C_67PER_4C_NEWSTRUCTURE	6	0.9952	0.9948-0.9966	10.042-10.044	中等
5_3C_54PER_4C_NEWSTRUCTURE	7	0.9954	0.9938-0.9974	10.042-10.044	中等
5_6C_36PER_4_3C_NEWSTRUCTURE	7	0.9941	0.9930-0.9968	10.041-10.045	中等
5_6C_19PER_4_6C_NEWSTRUCTURE	7	0.9939	0.9933-0.9953	10.042-10.044	中等
4_8C_80PER_4_8C	2	0.9854	0.9814-0.9894	10.058-10.068	短寿命
80PER_3_6C	2	0.9805	0.9679-0.9930	13.376-13.388	短寿命
3_7C_31PER_5_9C_NEWSTRUCTURE	2	0.9658	0.9601-0.9715	10.111-10.156	短寿命

注：策略类型按策略内 SOH 中位数与总体 25%/75% 分位阈值比较得到。 n 为满 200 圈电池数。

4.7 典型长寿命与短寿命策略

入选规则：以策略内 SOH_{200} 中位数为序；长寿命要求中位数 ≥ 0.996103 ，短寿命要求中位数 ≤ 0.993433 。

寿命最长策略：3_6C-80PER_3_6C，中位数 0.997734，范围 0.995232–1.000235，平均充电时间约 13.38 min ($n = 2$)。

典型长寿命策略：4_8C_80PER_4_8C_NEWSTRUCTURE，中位数 0.996334，但范围宽至 0.949733–0.996955，含一个明显偏低个体，解读时需结合离散程度。

寿命最短策略：3_7C_31PER_5_9C_NEWSTRUCTURE，中位数 0.965818，范围 0.960114–0.971521， $C_2 = 5.9\text{C}$ 为全样本最高第二阶段倍率。

典型短寿命策略：80PER_3_6C 与 4_8C_80PER_4_8C。

4.8 长、短寿命策略差异的初步解释

结合两阶段快充物理图像 [2,6,7]，可将差异归纳如下。

1. 高 SOC 区间高倍率。最短寿命策略在 $Q_1 = 31\%$ 之后仍以 5.9C 充电至 80% SOC。中高 SOC 下浓差极化、锂沉积风险与产热更显著，与题干“高 SOC 区间继续高倍率可能加剧老化”一致。
2. 低倍率、高 Q_1 并不自动等于长寿命。80PER_3_6C 充电时间长达约 13.4 min，但 SOH_{200} 仍偏低，说明“充得慢”不是充分保护条件，可能与长时间处于中高 SOC 或批次差异有关。
3. 同名义参数、不同标签不可等同。4_8C_80PER_4_8C 与 4_8C_80PER_4_8C_NEWSTRUCTURE 的 C_1, Q_1, C_2 相同，但前者被标为短寿命、后者中位数落入长寿命。二者平均充电时间分别约为 10.06 min 与 11.04 min，说明实际过程或批次并不相同。NEWSTRUCTURE 应理解为后期实验框架标签，不是“充电条件完全相同”。
4. 样本与窗口局限。若干策略仅 2 块满 200 圈电池；早期三策略与 NEWSTRUCTURE 策略跨批次对比应降级为现象观察。正式因果推断与显著性检验留待问题二 [8,9]。

待完成 / 框架占位

问题一仍可补充：（1）策略参数总表（每策略 C_1, Q_1, C_2 与 IR、温度均值）；（2）NEWSTRUCTURE 内部单独排序作为公平对比主结论；（3）9 策略平均 SOH-cycle 曲线。上述不影响已给出的清洗、分布与典型策略结论。

5 问题二：充电策略及其参数对寿命衰减的影响

5.1 分析口径与样本选择

赛题提供的循环记录最多至第 200 圈，且观测到的最低 SOH 仍约为 0.95，未观测到 SOH = 80% 的真实 EOL。因此，本节不把“寿命”解释为真实循环寿命，而采用第 200 圈平滑健康状态 SOH_{200} 及其衰减量

$$\text{Fade}_{200} = \text{SOH}_1 - \text{SOH}_{200} \quad (4)$$

作为早期健康保持程度的代理指标。

为降低不同数据批次及实验结构带来的混杂，主分析仅保留 `dataset_id=3`、非问题三测试集且完整记录至第 200 圈的电池，共 34 块。将策略名称中的 `_NEWSTRUCTURE` 后缀移除后，得到 6 个策略组；全部 40 块完整 200 圈样本仅用于敏感性参考。由于策略参数并非完全析因设计，以下结果均解释为样本内统计关联而非严格因果效应。

5.2 策略间差异检验

针对样本量不均衡、部分策略组很小的特点，对 SOH_{200} 采用 Kruskal–Wallis 秩和检验。结果为 $H = 12.3346$ 、 $p = 0.0305$ ，表明六个策略组的 SOH_{200} 分布在总体上存在差异。进一步进行 15 组 Mann–Whitney 两两比较，并采用 Holm 方法校正多重检验；校正后没有单一策略对达到 0.05 显著性水平。这说明总体差异主要由样本少、差异大的策略和组内离散性共同驱动，不宜将某一具体策略对写成已被充分验证的优劣结论。

表 4 主分析队列的第 200 圈 SOH 与衰减汇总

策略	样本数	$\overline{\text{SOH}_{200}}$	$\text{median}(\text{SOH}_{200})$	$\overline{\text{Fade}_{200}}$
4_8C_80PER_4_8C	5	0.987369	0.996961	0.001888
5C_67PER_4C	6	0.995523	0.995249	0.002686
5_3C_54PER_4C	7	0.995469	0.995451	0.002563
5_6C_19PER_4_6C	7	0.993987	0.993929	0.004392
5_6C_36PER_4_3C	7	0.994696	0.994186	0.003716
3_7C_31PER_5_9C	2	0.965908	0.965908	0.033751

图 5 显示，3_7C_31PER_5_9C 组在本窗口内的衰减量显著更大，但该组仅有 2 块电池；4_8C_80PER_4_8C 组包含一个明显较低的个体，故不能仅凭组均值或中位数进行排序。

5.3 参数关联与回归诊断

首先考察参数与 Fade_{200} 的 Spearman 相关。 Q_1 与衰减量呈负相关($\rho = -0.5889$, $p = 0.00025$)， C_1 与 C_2 的相关在本样本中均未达到 0.05 显著性水平。作为探索性补充，拟

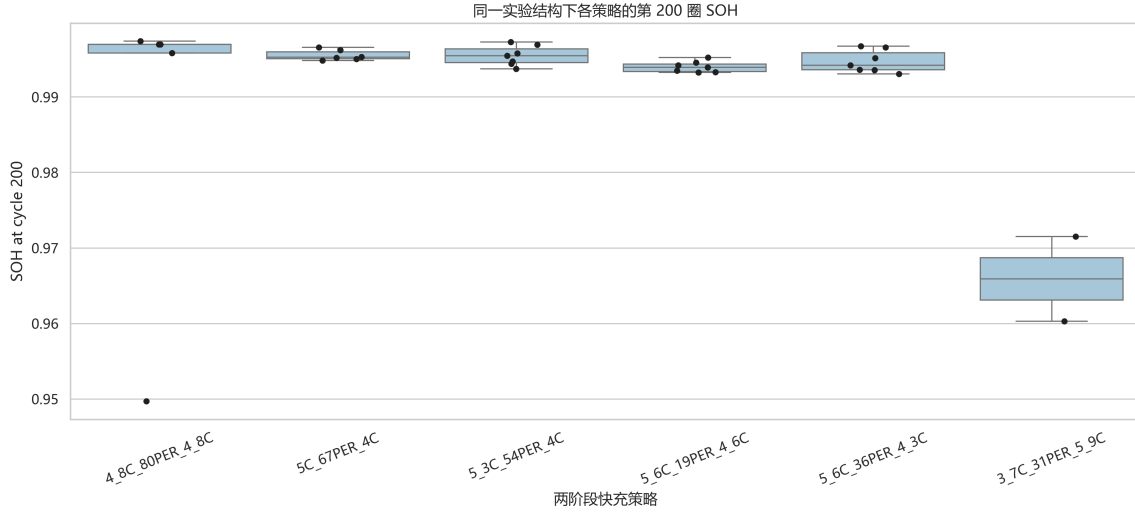


图 5 同一实验结构下，各策略第 200 圈 SOH 的箱线图与单体观测值

合

$$\text{Fade}_{200,i} = \beta_0 + \beta_1 C_{1,i} + \beta_2 Q_{1,i} + \beta_3 C_{2,i} + \varepsilon_i, \quad (5)$$

并采用 HC3 稳健标准误。模型调整 $R^2 = 0.9025$ ，但 C_1 与 C_2 的 VIF 分别为 31.88 和 33.09，表明严重共线性；因此该模型不能用来独立识别三项参数的边际因果贡献。特别地，式 (5) 中 C_1 的负系数和 Q_1 的负系数反映的是当前少量策略组合的共同变化，不能推出“提高 C_1 ”或“提高 Q_1 ”必然降低衰减。

结合图 6，本数据支持的稳妥表述是：在观测窗口内，低切换 SOC ($Q_1 = 31\%$) 后继续采用较高第二阶段倍率 ($C_2 = 5.9C$) 的策略出现了更快衰减的现象；这与高 SOC 区间更强极化、产热和锂沉积风险的机理相一致 [2,6,7]，但仍需独立、正交的重复实验来验证。

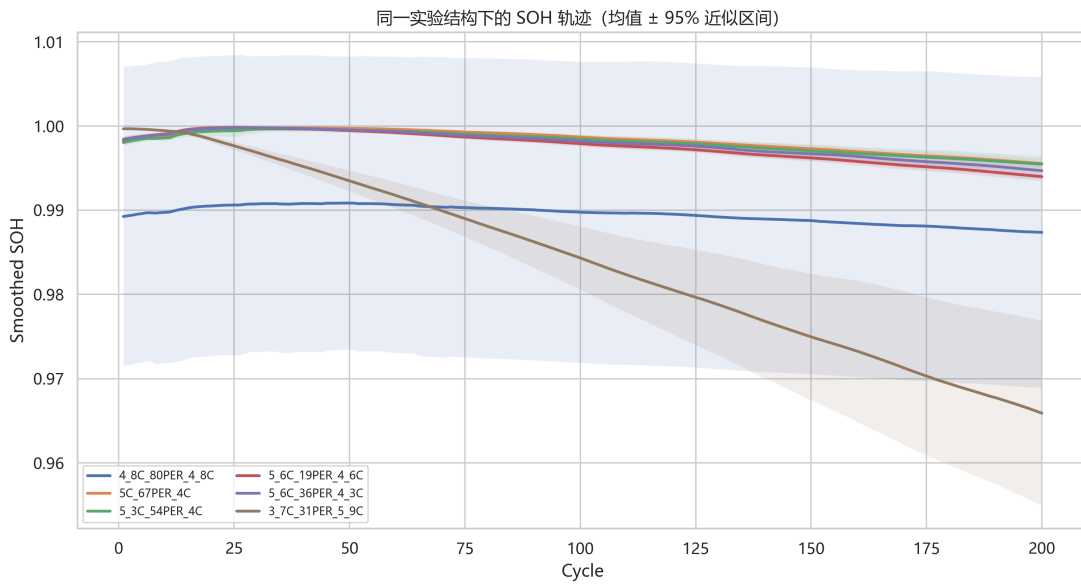


图 6 各策略的平滑 SOH 平均轨迹；阴影为均值的 95% 近似区间

5.4 纵向混合效应分析

为避免将同一电池 200 个循环点视为相互独立,进一步使用全部 1–200 圈的 $\text{SOH}_{\text{smooth}}$, 拟合固定效应为 $\text{cycle} \times \text{policy}$, 并为电池设置随机截距和随机循环斜率。以 3_7C_31PER_5_9C 为参考组、将循环次数缩放为每 100 圈后, 固定效应估计的参考组斜率为 -0.018 , 其余策略的交互项为 $+0.015 \sim +0.016$, 即各策略的估计平均斜率如表 5。

表 5 混合效应模型给出的平滑 SOH 平均斜率 (每 100 圈)

策略	固定效应斜率	样本数
3_7C_31PER_5_9C (参考)	-0.018	2
4_8C_80PER_4_8C	-0.002	5
5C_67PER_4C	-0.002	6
5_3C_54PER_4C	-0.002	7
5_6C_19PER_4_6C	-0.003	7
5_6C_36PER_4_3C	-0.002	7

该模型在数值上收敛, 但随机截距和随机斜率方差均被估计在边界附近, 表明在当前 34 块电池、且已使用平滑 SOH 的数据条件下, 电池间随机变异未被稳定识别。因此, 表 5 只用于量化图 6 中的轨迹现象; 不将 6800 条循环记录当作独立实验样本, 也不据此报告策略间的显著性优劣。

5.5 后续尝试方向

本题当前数据适合提出机制假设与早期衰减关联, 尚不足以完成强因果识别。后续可按以下路线推进:

1. **补足终止事件。**获取完整循环至 $\text{SOH} = 80\%$ 的记录后, 以真实 N_{EOL} 为响应, 采用加速失效时间模型或右删失生存模型; 当前所有样本在 150/200 圈时均未发生 EOL, 不能估计真实寿命分布。
2. **补做正交重复实验。**在相同批次、温度和单体条件下, 独立改变 C_1 、 Q_1 、 C_2 , 并针对高 SOC 区间的倍率设置配对对照; 每个组合应有足够重复, 才能区分参数效应与批次/个体差异。
3. **改进纵向模型。**以电池为重采样单位进行 cluster bootstrap, 比较线性混合模型、样条/GAMM 和分段退化模型; 同时将初始容量、温度、内阻及批次纳入协变量, 并采用留一策略交叉验证检验外推稳定性。
4. **重参数化与稳健推断。**用低/高 SOC 区间的加权倍率暴露量替代高度共线的 C_1, Q_1, C_2 , 结合岭回归或贝叶斯层级模型报告电池层面的置信区间, 而不是直接解读当前 OLS 的单个系数。

综上, 问题二的统计证据表明策略间早期健康保持程度存在总体差异; 高 SOC 区间高倍率策略的较快衰减值得在后续实验和问题四优化中作为风险约束考虑。但由于样

本小、参数共线且缺少真实 EOL 观测，本文不把第 200 圈 SOH 的差异外推为真实寿命差异。

6 问题三：基于早期循环的电池寿命预测

待完成 / 框架占位

本节为建模框架。9 块测试电池已在 `battery_summary.csv` 中由 `prediction_test=1` 标记，循环数据截至第 150 圈。预测表、误差指标与 EOL 外推结果待补入。

6.1 问题分析

实际中往往只能获得前期循环，无法等待完整老化。本题要求：用截至第 150 次循环的数据预测第 151–200 次循环 SOH，并进一步估计 $\text{SOH} = 80\%$ 的循环寿命；同时比较不同早期窗口长度，讨论策略信息与早期衰减特征的作用 [1, 10–13]。

6.2 特征提取（拟采用）

表 6 问题三拟提取的特征类别

类别	示例
策略参数	C_1, Q_1, C_2 及策略编码
衰减趋势	前 k 圈 SOH 斜率、曲率、幂律/指数拟合参数
过程量	充电时间、IR、温度的均值与变化率
分段差	$\Delta\text{SOH}_{1 \rightarrow 50}$ 、 $\Delta\text{SOH}_{50 \rightarrow 100}$ 、 $\Delta\text{SOH}_{100 \rightarrow 150}$

优先使用 `SOH_smooth`。若后续引入放电电压曲线类特征 $\Delta Q(V)$ ，需回源完整公开数据 [4, 10]。

6.3 预测模型（拟采用）

经验衰减模型（可解释）：

$$\text{SOH}(n) = 1 - a n^b \quad \text{或} \quad \text{SOH}(n) = a e^{-bn} + c, \quad (6)$$

用 1–150 圈拟合，外推 151–200 及 N_{EOL} 。

统计 / 机器学习模型：以早期统计特征预测后续 SOH 或衰减率，候选包括正则化线性模型、随机森林 / XGBoost、高斯过程；序列模型可试 LSTM [12]。验证建议：在 40 块非测试电池上做“截断 150 圈 → 预测 151–200”的交叉验证，再对 9 块测试电池给出最终预报，避免同策略信息泄漏。

6.4 评价指标与窗口比较

对 151–200 圈报告 RMSE、MAE 与 R^2 ；对 N_{EOL} 报告外推圈数。比较输入窗口 $k \in \{50, 100, 150\}$ 的精度差异。消融：有/无 (C_1, Q_1, C_2) 。

待填：9 块测试电池的 SOH 预测曲线、误差表、EOL 点估计。

7 问题四：兼顾充电时间与寿命衰减的策略优化

待完成 / 框架占位

本节为建模框架。优化应优先在已有 9 种实验策略内比较；若连续优化，参数须限制在实验数据合理邻域并说明依据 [14–17]。

7.1 决策变量与目标

决策变量 $x = (C_1, Q_1, C_2)$ 。目标为在较短充电时间下降低衰减速率或延长寿命：

$$\min_x F(x) = \begin{bmatrix} T_{\text{charge}}(x) \\ \text{DecayRate}(x) \end{bmatrix} \quad \text{或} \quad \min_x w_1 T_{\text{charge}}(x) + w_2 \text{DecayRate}(x). \quad (7)$$

7.2 充电时间模型（拟采用）

分段恒流近似

$$T_{\text{charge}} \approx \frac{Q_1}{C_1} + \frac{0.8 - Q_1}{C_2} + T_{\text{CV}}, \quad (8)$$

并用 `mean_chargetime` 回归校准。问题一散点图已显示充电时间与策略分组相关，可用于检验该近似。

7.3 寿命衰减模型

沿用问题二回归或问题三预测模型，得到 $\text{DecayRate}(x)$ 或 $\text{SOH}_{200}(x)$ 。在已有策略上先做离散 Pareto 比较，再考虑邻域连续搜索。

7.4 约束与邻域

建议参数盒约束取自样本极值（以最终校核为准）： $C_1 \in [3.6, 5.6]$ ， $Q_1 \in [19, 80]$ ， $C_2 \in [3.6, 5.9]$ 。超出实验凸包的组合须标明外推风险。

7.5 推荐策略与对比（待填）

给出推荐 (C_1^*, Q_1^*, C_2^*) ，并与问题一的典型长寿命策略 `3_6C-80PER_3_6C_4_8C_80PER_4_8C_NEWST` 及短寿命策略 `3_7C_31PER_5_9C_NEWSTRUCTURE` 对比充电时间与寿命代理。说明适用范

围（LFP、两阶段快充、早期循环窗口）与不可外推情形。

8 结论与展望

已完成结果（仓库现有成果）

问题一已形成的结论：

- 循环数据经 IQR 清洗后可用于策略对比；原始数据文件未改写。
- 在满 200 圈的 40 块电池中，第 200 次循环 SOH 均值为 0.9914，尚未达到 80% EOL。
- 按策略内 SOH 中位数，最长寿命策略为 $3.6C(80\%) - 3.6C$ ，最短为 $3.7C(31\%) - 5.9C$ （NEWSTRUCTURE）。
- 高 SOC 区间维持高 C_2 的策略更容易出现更快衰减；同参数不同标签策略不可直接等同。

已完成结果（仓库现有成果）

问题二结论：

- 在同一实验结构的 34 块完整样本上，六种策略的 SOH_{200} 分布存在总体差异（Kruskal-Wallis: $H = 12.3346, p = 0.0305$ ）。
- Holm 校正后的两两比较均未显著，故不对具体策略作确定性的优劣排序。
- Q_1 与 200 圈衰减量呈显著负 Spearman 相关，但 C_1 、 C_2 共线性严重；参数效应仅解释为样本内关联。

待完成 / 框架占位

补充问题三预测精度与 EOL 估计、问题四推荐策略及与长短寿命策略的对比；讨论模型局限与可推广范围。

参考文献

参考文献

- [1] Kristen A. Severson, Peter M. Attia, Norman Jin, Nicholas Perkins, Benben Jiang, Zi Yang, Michael H. Chen, Muratahan Aykol, Patrick K. Herring, Dimitrios Fraggedakis, Martin Z. Bazant, Stephen J. Harris, William C. Chueh, and Richard D.

- Braatz. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4(5):383–391, 2019. 本题主数据集来源；问题 1-4 必引。
- [2] Yayuan Liu, Yangying Zhu, and Yi Cui. Challenges and opportunities towards fast-charging battery materials. *Nature Energy*, 4:540–550, 2019. 快充机理综述；问题 1 机制解释。
- [3] Shabbir Ahmed, Ira Bloom, Andrew N. Jansen, Tanvir R. Tanim, Eric J. Dufek, Ahmad Pesaran, et al. Enabling fast charging – A battery technology gap assessment. *Journal of Power Sources*, 367:250–262, 2017. 快充技术缺口评估；问题 1/2 背景。
- [4] MIT–Stanford–TRI battery dataset. <https://data.matr.io/1/>. Severson 2019 配套公开数据；本题主数据。
- [5] Gregory L. Plett. *Battery Management Systems, Volume I: Battery Modeling*. Artech House, 2015. SOH/建模基础。
- [6] Matthieu Dubarry and Bor Yann Liaw. Identify capacity fading mechanism in a commercial LiFePO₄ cell. *Journal of Power Sources*, 194:541–549, 2009. LFP 衰减机理；问题 1/2。
- [7] Y. Ma et al. An aging-optimized state-of-charge-controlled multi-stage constant current (MCC) fast charging algorithm for commercial Li-ion battery based on three-electrode measurements. *Batteries*, 10(8):267, 2024. 多段恒流快充；问题 2/4。
- [8] Michael H. Kutner, Christopher J. Nachtsheim, John Neter, and William Li. *Applied Linear Statistical Models*. McGraw-Hill, 5 edition, 2005. 问题 2 ANOVA/回归。
- [9] Journal of The Electrochemical Society contributors. Fast charging of lithium-ion batteries while accounting for degradation and cell-to-cell variability. *Journal of The Electrochemical Society*, 2024. 快充、退化与单体差异；问题 2/4。
- [10] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, et al. Statistical learning for accurate and interpretable battery lifetime prediction. *Journal of The Electrochemical Society*, 2021. 问题 3 特征工程与可解释预测。
- [11] Alexis Geslin et al. Commentary on charging features for battery lifetime prediction. *Joule*, 2023. Charging-condition features and lifetime prediction; Problem 2/3.
- [12] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries. *IEEE Transactions on Vehicular Technology*, 67(7):5695–5705, 2018. LSTM-RNN and Monte Carlo probabilistic RUL prediction.

- [13] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Lithium-ion battery remaining useful life prediction with box-cox transformation and monte carlo simulation. *IEEE Transactions on Industrial Electronics*, 66(2):1585–1597, 2019. Box-Cox transformation and Monte Carlo simulation for online RUL prediction.
- [14] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, Todor M. Markov, Yang Liao, Michael H. Chen, Bryan Cheong, Nicholas Perkins, Zi Yang, Patrick K. Herring, Muratahan Aykol, Stephen J. Harris, Richard D. Braatz, Stefano Ermon, and William C. Chueh. Closed-loop optimization of fast-charging protocols for batteries with machine learning. *Nature*, 578:397–402, 2020. 问题 3 早期预测 + 问题 4 贝叶斯优化快充协议.
- [15] Gregory L. Plett and M. Scott Trimboli. *Battery Management Systems, Volume III: Physics-Based Methods*. Artech House, 2022. 物理模型与快充优化; 问题 4.
- [16] Peter I. Frazier. A tutorial on Bayesian optimization, 2018. 问题 4 贝叶斯优化方法.
- [17] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001. 问题 4 多目标优化 / NSGA-II.

A 支撑材料文件列表

按赛制，支撑材料单独打包为压缩文件（RAR/ZIP，不超过 20 MB），本附录列出其内容。赛题原始数据 `data/battery_summary.csv`、`data/cycle_train.csv` 不作为“自主查阅数据”重复提交，但建模程序依赖它们运行。

表 7 支撑材料拟包含的文件

路径	说明
<code>src/A/task1_pipeline.py</code>	问题一完整可运行流水线
<code>src/A/SOH_plot.py</code>	SOH 曲线绘制模块
<code>A/问题一/AGENTS.md</code>	问题一任务说明（提示词）
<code>A/问题一/log_1.md</code> 、 <code>log_2.md</code>	运行日志
<code>output/A/cycle_train_cleaned.csv</code>	IQR 清洗后循环数据
<code>output/A/charge_policy.csv</code>	电池-策略表
<code>output/A/battery_SOH_charge.csv</code>	200 圈 SOH 与充电时间
<code>output/A/policy.csv</code>	策略汇总与寿命标签
<code>output/A/doc/1.md</code>	典型长/短寿命策略说明
<code>output/A/*.png</code>	问题一图件
<code>output/A/1/</code>	问题一第一轮归档
<code>references/</code>	参考文献导读与 BibTeX
<code>AI 使用记录/</code>	AI 使用详情底稿
<code>paper/</code>	本 LaTeX 工程

问题二至问题四的脚本、中间表与预测结果待生成后追加本表。若最终提交时上述文件均已纳入压缩包，则与论文结论对应；源程序运行方式见附录 B。

B 建模源程序

本节给出问题一全部可运行源代码。运行环境：Python 3，依赖 numpy、pandas、matplotlib。在项目根目录执行：

```
1 python src/A/task1_pipeline.py
```

可选参数 `--iqr-factor`（默认 1.5）。单独绘制指定电池 SOH 曲线：

```
1 python src/A/SOH_plot.py --battery-id 1
```

待完成 / 框架占位

问题二–四源程序完成后，以同样方式用 `\lstinputlisting` 追加完整代码，确保运行结果与正文图表一致。

B.1 SOH_plot.py

Listing 1 SOH 曲线绘制脚本

```
1 """绘制指定电池的 SOH 曲线。"""
2
3 from __future__ import annotations
4
5 import argparse
6 from pathlib import Path
7
8 import matplotlib
9
10 matplotlib.use("Agg")
11
12 import matplotlib.font_manager as font_manager
13 import matplotlib.pyplot as plt
14 import numpy as np
15 import pandas as pd
16
17
18 PROJECT_ROOT = Path(__file__).resolve().parents[2]
19
20
21 def configure_chinese_font() -> str:
```

```

22     """选择系统中可用的中文字体，并返回字体名称。"""
23     installed = {font.name for font in font_manager.fontManager.ttflist}
24     candidates = [
25         "Microsoft YaHei",
26         "SimHei",
27         "Noto Sans CJK SC",
28         "Source Han Sans SC",
29         "Arial Unicode MS",
30     ]
31     selected = next((name for name in candidates if name in installed), "DejaVu
        Sans")
32     plt.rcParams["font.sans-serif"] = [selected, "DejaVu Sans"]
33     plt.rcParams["axes.unicode_minus"] = False
34     return selected
35
36
37 def plot_battery_soh(
38     cycles: pd.DataFrame,
39     battery_id: int,
40     output_path: Path,
41 ) -> None:
42     """绘制指定电池的平滑 SOH 曲线。"""
43     required = {"battery_id", "cycle", "SOH_smooth"}
44     missing = required.difference(cycles.columns)
45     if missing:
46         raise ValueError(f"循环数据缺少字段: {sorted(missing)}")
47
48     battery = cycles.loc[cycles["battery_id"] == battery_id].sort_values("cycle"
        )
49     if battery.empty:
50         raise ValueError(f"未找到 battery_id={battery_id} 的数据")
51
52     configure_chinese_font()
53     fig, ax = plt.subplots(figsize=(10, 6))
54     ax.plot(
55         battery["cycle"],
56         battery["SOH_smooth"],
57         color="#4C78A8",
58         linewidth=2.2,
59         label="平滑 SOH (SOH_smooth) ",
60     )
61     ax.set_title(f"{battery_id} 号电池 SOH 曲线")
62     ax.set_xlabel("循环次数")
63     ax.set_ylabel("电池健康状态 (SOH) ")

```

```

64 ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
65 ax.legend(loc="upper right")
66 fig.tight_layout()
67 output_path.parent.mkdir(parents=True, exist_ok=True)
68 fig.savefig(output_path, dpi=300, bbox_inches="tight")
69 plt.close(fig)
70
71
72 def plot_all_battery_soh(cycles: pd.DataFrame, output_path: Path) -> None:
73     """用蓝到红渐变绘制全部电池的平滑 SOH 曲线。"""
74     required = {"battery_id", "cycle", "SOH_smooth"}
75     missing = required.difference(cycles.columns)
76     if missing:
77         raise ValueError(f"循环数据缺少字段: {sorted(missing)}")
78
79     battery_ids = sorted(int(value) for value in cycles["battery_id"].unique())
80     if not battery_ids:
81         raise ValueError("循环数据中没有电池记录")
82
83     configure_chinese_font()
84     fig, ax = plt.subplots(figsize=(14, 8))
85     colormap = matplotlib.colormaps.get_cmap("coolwarm")
86     colors = colormap(np.linspace(0, 1, len(battery_ids)))
87     for color, battery_id in zip(colors, battery_ids):
88         battery = cycles.loc[cycles["battery_id"] == battery_id].sort_values("
            cycle")
89         ax.plot(
90             battery["cycle"],
91             battery["SOH_smooth"],
92             color=color,
93             linewidth=1.05,
94             alpha=0.78,
95         )
96
97     ax.set_title("全部电池平滑 SOH 曲线")
98     ax.set_xlabel("循环次数")
99     ax.set_ylabel("电池健康状态 (SOH_smooth)")
100     ax.grid(True, color="#D9D9D9", linewidth=0.6, alpha=0.6)
101     ax.legend(
102         handles=[
103             plt.Line2D([0], [0], color=colors[0], linewidth=2, label=f"低编号: 电
                池 {battery_ids[0]}"),
104             plt.Line2D([0], [0], color=colors[-1], linewidth=2, label=f"高编号: 电
                池 {battery_ids[-1]}"),

```

```

105     ],
106     loc="upper right",
107     title="颜色由蓝到红",
108 )
109 scalar_mappable = plt.cm.ScalarMappable(
110     cmap=colormap,
111     norm=plt.Normalize(vmin=battery_ids[0], vmax=battery_ids[-1]),
112 )
113 scalar_mappable.set_array([])
114 colorbar = fig.colorbar(scalar_mappable, ax=ax, pad=0.015)
115 colorbar.set_label("battery_id")
116 fig.tight_layout()
117 output_path.parent.mkdir(parents=True, exist_ok=True)
118 fig.savefig(output_path, dpi=300, bbox_inches="tight")
119 plt.close(fig)
120
121
122 def parse_args() -> argparse.Namespace:
123     parser = argparse.ArgumentParser(description="绘制指定电池的 SOH 曲线")
124     parser.add_argument("--battery-id", type=int, default=1, help="电池编号，默认
125         为 1")
126     parser.add_argument(
127         "--input",
128         type=Path,
129         default=PROJECT_ROOT / "output" / "A" / "cycle_train_cleaned.csv",
130         help="循环数据 CSV 路径",
131     )
132     parser.add_argument(
133         "--output",
134         type=Path,
135         default=PROJECT_ROOT / "output" / "A" / "SOH_1_example.png",
136         help="输出图像路径",
137     )
138     return parser.parse_args()
139
140 def main() -> None:
141     args = parse_args()
142     cycles = pd.read_csv(args.input)
143     plot_battery_soh(cycles, args.battery_id, args.output)
144     print(f"已生成: {args.output}")
145
146
147 if __name__ == "__main__":

```

```
148 main()
```

B.2 task1_pipeline.py

Listing 2 问题一分析流水线

```
1  """问题一：数据清洗、策略提取、200 次循环汇总与绘图。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from collections import Counter
7  from datetime import datetime
8  from pathlib import Path
9
10 import matplotlib
11
12 matplotlib.use("Agg")
13
14 import matplotlib.pyplot as plt
15 from matplotlib.patches import Patch
16 import numpy as np
17 import pandas as pd
18
19 from SOH_plot import configure_chinese_font, plot_all_battery_soh,
20     plot_battery_soh
21
22 PROJECT_ROOT = Path(__file__).resolve().parents[2]
23 MEASUREMENT_COLUMNS = ["capacity", "SOH", "SOH_smooth", "chargetime", "IR", "
24     Tavg"]
25
26 def normalize_policy_name(policy: str) -> str:
27     """按第三次迭代规则合并策略并删除 NEWSTRUCTURE 后缀。"""
28     normalized = str(policy).strip()
29     direct_mapping = {
30         "80PER_3_6C": "3_6C-80PER_3_6C",
31         "3_6C-80PER_3_6C": "3_6C-80PER_3_6C",
32         "4_8C_80PER_4_8C_NEWSTRUCTURE": "4_8C_80PER_4_8C",
33         "4_8C_80PER_4_8C": "4_8C_80PER_4_8C",
34     }
35     if normalized in direct_mapping:
36         return direct_mapping[normalized]
```

```

37     upper = normalized.upper()
38     if upper.endswith("_NEWSTRUCTURE"):
39         normalized = normalized[: -len("_NEWSTRUCTURE")]
40     return normalized
41
42
43 def normalize_policies(
44     summary: pd.DataFrame,
45     cycles: pd.DataFrame,
46 ) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, str]]:
47     """在处理副本中统一两个数据表的策略名称，原始数据保持不变。"""
48     normalized_summary = summary.copy()
49     normalized_cycles = cycles.copy()
50     original_names = sorted(str(value) for value in summary["policy"].unique())
51     mapping = {name: normalize_policy_name(name) for name in original_names}
52     normalized_summary["policy"] = normalized_summary["policy"].map(mapping)
53     normalized_cycles["policy"] = normalized_cycles["policy"].map(mapping)
54     if normalized_summary["policy"].isna().any() or normalized_cycles["policy"].
55         isna().any():
56         raise ValueError("策略统一命名过程中产生空值")
57     return normalized_summary, normalized_cycles, mapping
58
59 def validate_inputs(summary: pd.DataFrame, cycles: pd.DataFrame) -> None:
60     summary_required = {"battery_id", "policy", "initial_capacity", "
61         prediction_test"}
62     cycle_required = {
63         "battery_id",
64         "cycle",
65         "capacity",
66         "SOH",
67         "SOH_smooth",
68         "chargetime",
69         "IR",
70         "Tavg",
71         "policy",
72     }
73     summary_missing = summary_required.difference(summary.columns)
74     cycle_missing = cycle_required.difference(cycles.columns)
75     if summary_missing or cycle_missing:
76         raise ValueError(
77             f"输入字段不完整; battery_summary 缺少 {sorted(summary_missing)}, "
78             f"cycle_train 缺少 {sorted(cycle_missing)}"
79         )

```

```

79     if summary["battery_id"].duplicated().any():
80         raise ValueError("battery_summary.csv 中 battery_id 不唯一")
81     if cycles.duplicated(["battery_id", "cycle"]).any():
82         raise ValueError("cycle_train.csv 中存在重复的 battery_id + cycle")
83     if set(summary["battery_id"]) != set(cycles["battery_id"]):
84         raise ValueError("两个输入文件的 battery_id 集合不一致")
85
86     policy_check = cycles.merge(
87         summary[["battery_id", "policy"]],
88         on="battery_id",
89         suffixes=("_cycle", "_summary"),
90         validate="many_to_one",
91     )
92     if (policy_check["policy_cycle"] != policy_check["policy_summary"]).any():
93         raise ValueError("cycle_train.csv 与 battery_summary.csv 的策略字段不一致")
94
95
96     def iqr_clean_by_battery(
97         cycles: pd.DataFrame,
98         columns: list[str],
99         factor: float = 1.5,
100     ) -> tuple[pd.DataFrame, list[dict[str, float | int | str]]]:
101         """逐电池、逐字段进行 IQR 检测，并用最近前后正常值的均值替换。"""
102         cleaned = cycles.sort_values(["battery_id", "cycle"]).copy()
103         audit: list[dict[str, float | int | str]] = []
104
105         for battery_id, group in cycles.sort_values(["battery_id", "cycle"]).groupby(
106             "battery_id"):
107             for column in columns:
108                 values = group[column].to_numpy(dtype=float)
109                 finite_values = values[np.isfinite(values)]
110                 if finite_values.size == 0:
111                     continue
112
113                 q1, q3 = np.quantile(finite_values, [0.25, 0.75])
114                 iqr = q3 - q1
115                 if np.isclose(iqr, 0.0):
116                     continue
117
118                 lower = q1 - factor * iqr
119                 upper = q3 + factor * iqr
120                 outlier_mask = (~np.isfinite(values)) | (values < lower) | (values >
121                     upper)
122                 normal_positions = np.flatnonzero(~outlier_mask)

```

```

120
121     for position in np.flatnonzero(outlier_mask):
122         left_candidates = normal_positions[normal_positions < position]
123         right_candidates = normal_positions[normal_positions > position]
124         neighbors: list[float] = []
125         if left_candidates.size:
126             neighbors.append(float(values[left_candidates[-1]]))
127         if right_candidates.size:
128             neighbors.append(float(values[right_candidates[0]]))
129         if not neighbors:
130             continue
131
132         replacement = float(np.mean(neighbors))
133         row_index = group.index[position]
134         cleaned.at[row_index, column] = replacement
135         audit.append(
136             {
137                 "battery_id": int(battery_id),
138                 "cycle": int(group.iloc[position]["cycle"]),
139                 "column": column,
140                 "original": float(values[position]),
141                 "replacement": replacement,
142                 "lower_bound": float(lower),
143                 "upper_bound": float(upper),
144             }
145         )
146
147     return cleaned.sort_values(["battery_id", "cycle"]), audit
148
149
150 def build_policy_table(summary: pd.DataFrame) -> pd.DataFrame:
151     return (
152         summary[["battery_id", "policy"]]
153         .sort_values("battery_id")
154         .reset_index(drop=True)
155     )
156
157
158 def build_cycle_200_summary(
159     cleaned_cycles: pd.DataFrame,
160     policy_table: pd.DataFrame,
161 ) -> pd.DataFrame:
162     max_cycle = cleaned_cycles.groupby("battery_id")["cycle"].max()
163     eligible_ids = max_cycle[max_cycle == 200].index

```



```

164     eligible = cleaned_cycles[cleaned_cycles["battery_id"].isin(eligible_ids)]
165
166     cycle_200 = eligible.loc[eligible["cycle"] == 200, ["battery_id", "SOH"]]
167     if cycle_200["battery_id"].duplicated().any() or len(cycle_200) != len(
168         eligible_ids):
169         raise ValueError("达到 200 次循环的电池没有且仅有一条 cycle=200 记录")
170
171     charge_mean = (
172         eligible.groupby("battery_id", as_index=False)["chargetime"]
173         .mean()
174         .rename(columns={"chargetime": "charge_time_mean"})
175     )
176     result = (
177         cycle_200.merge(policy_table, on="battery_id", validate="one_to_one")
178         .merge(charge_mean, on="battery_id", validate="one_to_one")
179         [["battery_id", "policy", "SOH", "charge_time_mean"]]
180         .sort_values("battery_id")
181         .reset_index(drop=True)
182     )
183     return result
184
185 def build_policy_summary(
186     data: pd.DataFrame,
187     policy_order: list[str],
188     critical_soh_low: float,
189     critical_soh_high: float,
190     summary: pd.DataFrame,
191     cleaned_cycles: pd.DataFrame,
192 ) -> pd.DataFrame:
193     """汇总统一策略的最终 SOH、充电时间和其他实验参数。"""
194     classified = data.assign(
195         lifetime_group=np.select(
196             [data["SOH"] >= critical_soh_high, data["SOH"] <= critical_soh_low],
197             ["长寿命", "短寿命"],
198             default="中等寿命",
199         )
200     )
201     grouped = classified.groupby("policy", sort=False)
202     result = grouped.agg(
203         battery_count=("battery_id", "size"),
204         SOH_min=("SOH", "min"),
205         SOH_max=("SOH", "max"),
206         SOH_mean=("SOH", "mean"),

```

```

207     SOH_median=("SOH", "median"),
208     charge_time_mean=("charge_time_mean", "mean"),
209     charge_time_min=("charge_time_mean", "min"),
210     charge_time_max=("charge_time_mean", "max"),
211 ).reindex(policy_order)
212 result["battery_ids"] = grouped["battery_id"].apply(
213     lambda values: ",".join(str(int(value)) for value in sorted(values))
214 ).reindex(policy_order)
215
216 eligible_ids = set(int(value) for value in data["battery_id"])
217 battery_cycle_metrics = (
218     cleaned_cycles.loc[cleaned_cycles["battery_id"].isin(eligible_ids)]
219     .groupby(["battery_id", "policy"], as_index=False)
220     .agg(IR_mean=("IR", "mean"), Tavg_mean=("Tavg", "mean"))
221 )
222 cycle_metrics = battery_cycle_metrics.groupby("policy").agg(
223     IR_mean=("IR_mean", "mean"),
224     Tavg_mean=("Tavg_mean", "mean"),
225 ).reindex(policy_order)
226 result = result.join(cycle_metrics)
227
228 eligible_summary = summary.loc[summary["battery_id"].isin(eligible_ids)]
229 capacity_metrics = eligible_summary.groupby("policy").agg(
230     initial_capacity_mean=("initial_capacity", "mean"),
231     initial_capacity_min=("initial_capacity", "min"),
232     initial_capacity_max=("initial_capacity", "max"),
233 ).reindex(policy_order)
234 result = result.join(capacity_metrics)
235
236 def parameter_values(series: pd.Series) -> str:
237     values = [f"{float(value):g}" for value in sorted(series.dropna().unique
238         ())]
239     if series.isna().any():
240         values.append("缺失")
241     return "/".join(values)
242
243 for column in ["C1", "Q1", "C2"]:
244     if column in eligible_summary.columns:
245         result[f"{column}_values"] = (
246             eligible_summary.groupby("policy")[column].apply(parameter_values)
247             .reindex(policy_order)
248         )
249 group_counts = (
250     classified.groupby(["policy", "lifetime_group"])

```

```

249         .size()
250         .unstack(fill_value=0)
251         .reindex(policy_order, fill_value=0)
252     )
253     for group_name, column_name in [
254         ("长寿命", "long_battery_count"),
255         ("中等寿命", "middle_battery_count"),
256         ("短寿命", "short_battery_count"),
257     ]:
258         result[column_name] = group_counts.get(group_name, pd.Series(0, index=
                result.index)).astype(int)
259
260     result["lifetime_label"] = np.select(
261         [result["SOH_median"] >= critical_soh_high, result["SOH_median"] <=
                critical_soh_low],
262         ["长寿命策略", "短寿命策略"],
263         default="中等寿命策略",
264     )
265     max_median = result["SOH_median"].max()
266     min_median = result["SOH_median"].min()
267     result["longest_strategy"] = np.where(np.isclose(result["SOH_median"],
                max_median), "是", "否")
268     result["shortest_strategy"] = np.where(np.isclose(result["SOH_median"],
                min_median), "是", "否")
269     result["SOH_range"] = result.apply(lambda row: f"{row['SOH_min']:.6f} - {row
                ['SOH_max']:.6f}", axis=1)
270     result["charge_time_range"] = result.apply(
271         lambda row: f"{row['charge_time_min']:.6f} - {row['charge_time_max']:.6f}"
                , axis=1
272     )
273     return result.reset_index()
274
275
276 def save_soh_histogram(
277     data: pd.DataFrame,
278     output_path: Path,
279     critical_soh_low: float,
280     critical_soh_high: float,
281 ) -> None:
282     values = data["SOH"].to_numpy(dtype=float)
283     bin_edges = np.histogram_bin_edges(values, bins="fd")
284     if len(bin_edges) < 7:
285         bin_edges = np.linspace(values.min(), values.max(), 8)
286

```

```

287 fig, ax = plt.subplots(figsize=(10, 6))
288 ax.hist(
289     values,
290     bins=bin_edges,
291     color="#4C78A8",
292     edgecolor="white",
293     linewidth=1.0,
294     label="电池数量",
295 )
296 ax.axvline(
297     critical_soh_high,
298     color="#D62728",
299     linestyle="--",
300     linewidth=2,
301     label=f"上四分位线 (Q3) = {critical_soh_high:.4f}",
302 )
303 ax.axvline(
304     critical_soh_low,
305     color="#1F77B4",
306     linestyle="--",
307     linewidth=2,
308     label=f"下四分位线 (Q1) = {critical_soh_low:.4f}",
309 )
310 margin = max((values.max() - values.min()) * 0.08, 0.003)
311 ax.set_xlim(values.min() - margin, values.max() + margin)
312 ax.set_title("第 200 次循环 SOH 分布")
313 ax.set_xlabel("第 200 次循环的 SOH")
314 ax.set_ylabel("电池数量")
315 ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
316 ax.legend(loc="upper right")
317 fig.tight_layout()
318 fig.savefig(output_path, dpi=300, bbox_inches="tight")
319 plt.close(fig)
320
321
322 def save_soh_charge_scatter(
323     data: pd.DataFrame,
324     output_path: Path,
325     policy_order: list[str],
326 ) -> None:
327     fig, ax = plt.subplots(figsize=(13, 8))
328     color_map = matplotlib.colormaps.get_cmap("tab10")
329     colors = color_map(np.linspace(0, 1, max(len(policy_order), 2)))
330     for color, policy in zip(colors, policy_order):

```

```

331     group = data.loc[data["policy"] == policy]
332     if group.empty:
333         continue
334     ax.scatter(
335         group["SOH"],
336         group["charge_time_mean"],
337         s=64,
338         color=color,
339         edgecolors="white",
340         linewidths=0.8,
341         alpha=0.88,
342         label=policy,
343     )
344     ax.set_title("第 200 次循环 SOH 与平均充电时间")
345     ax.set_xlabel("第 200 次循环的 SOH")
346     ax.set_ylabel("平均充电时间 (min) ")
347     ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
348     ax.legend(loc="upper right", fontsize=8, framealpha=0.92, title="充电策略",
349             title_fontsize=9)
350     fig.tight_layout()
351     fig.savefig(output_path, dpi=300, bbox_inches="tight")
352     plt.close(fig)
353
354 def save_policy_boxplot(
355     data: pd.DataFrame,
356     output_path: Path,
357     critical_soh_low: float,
358     critical_soh_high: float,
359     policy_order: list[str],
360 ) -> None:
361     available_order = [policy for policy in policy_order if policy in set(data["
362         policy"])]
363     groups = [data.loc[data["policy"] == policy, "SOH"].to_numpy() for policy in
364         available_order]
365
366     fig, ax = plt.subplots(figsize=(16, 8))
367     box = ax.boxplot(
368         groups,
369         tick_labels=available_order,
370         patch_artist=True,
371         widths=0.62,
372         medianprops={"color": "#1F1F1F", "linewidth": 1.5},
373         whiskerprops={"color": "#555555"},

```

```

372     capprops={"color": "#555555"},
373     flierprops={
374         "marker": "o",
375         "markerfacecolor": "white",
376         "markeredgecolor": "#4C78A8",
377         "markersize": 5,
378     },
379 )
380 for patch in box["boxes"]:
381     patch.set_facecolor("#9ECAE1")
382     patch.set_edgecolor("#4C78A8")
383
384 ax.axhline(
385     critical_soh_high,
386     color="#D62728",
387     linestyle="--",
388     linewidth=2,
389 )
390 ax.axhline(
391     critical_soh_low,
392     color="#1F77B4",
393     linestyle="--",
394     linewidth=2,
395 )
396 all_values = data["SOH"].to_numpy(dtype=float)
397 margin = max((all_values.max() - all_values.min()) * 0.08, 0.003)
398 ax.set_ylim(all_values.min() - margin, all_values.max() + margin)
399 ax.set_title("不同充电策略下的第 200 次循环 SOH 分布")
400 ax.set_xlabel("充电策略")
401 ax.set_ylabel("使用寿命指标: 第 200 次循环的 SOH")
402 ax.tick_params(axis="x", labelrotation=40)
403 for label in ax.get_xticklabels():
404     label.set_horizontalalignment("right")
405 ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
406 legend_handles = [
407     Patch(facecolor="#9ECAE1", edgecolor="#4C78A8", label="策略内 SOH 分布"),
408     plt.Line2D(
409         [0], [0], color="#D62728", linestyle="--", linewidth=2,
410         label=f"上四分位线 (Q3)={critical_soh_high:.4f}",
411     ),
412     plt.Line2D(
413         [0], [0], color="#1F77B4", linestyle="--", linewidth=2,
414         label=f"下四分位线 (Q1)={critical_soh_low:.4f}",
415     ),

```

```

416 ]
417 ax.legend(handles=legend_handles, loc="upper right")
418 fig.tight_layout()
419 fig.savefig(output_path, dpi=300, bbox_inches="tight")
420 plt.close(fig)
421
422
423 def write_typical_policy_doc(
424     doc_path: Path,
425     policy_summary: pd.DataFrame,
426     critical_soh_low: float,
427     critical_soh_high: float,
428 ) -> None:
429     long_policies = policy_summary.loc[policy_summary["lifetime_label"] == "长寿命策略"]
430     short_policies = policy_summary.loc[policy_summary["lifetime_label"] == "短寿命策略"]
431     longest = policy_summary.loc[policy_summary["longest_strategy"] == "是"].iloc[0]
432     shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是"].iloc[0]
433
434     lines = [
435         "# 典型长寿命与短寿命充电策略",
436         "",
437         "本分析使用第 200 次循环的 SOH 作为早期使用寿命代理指标。",
438         f"长寿命阈值为样本 SOH 的 75% 分位数 ({critical_soh_high:.6f}) , ",
439         f"短寿命阈值为 25% 分位数 ({critical_soh_low:.6f})。策略类型按策略内 SOH 中位数判定。",
440         "",
441         "## 关键结论",
442         "",
443         f"- **寿命最长策略: `{longest['policy']}`**, SOH 中位数 {longest['SOH_median']:.6f}, 范围 {longest['SOH_range']}。",
444         f"- **寿命最短策略: `{shortest['policy']}`**, SOH 中位数 {shortest['SOH_median']:.6f}, 范围 {shortest['SOH_range']}。",
445         "",
446         "## 典型长寿命策略",
447         "",
448         "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 |",
449         "| ---| ---: | ---: | ---| ---| ---|",
450     ]
451     for _, row in long_policies.sort_values("SOH_median", ascending=False).iterrows():

```

```

452     mark = "寿命最长" if row["longest_strategy"] == "是" else "典型长寿命"
453     lines.append(
454         f"| `{row['policy']}` | {int(row['battery_count'])} | {row['SOH_median']:.6f} | "
455         f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** | "
456     )
457     lines.extend(
458         [
459             "",
460             "## 典型短寿命策略",
461             "",
462             "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 | "
463             ,
464             "| ---|---:|---:|---|---|---|",
465         ]
466     )
467     for _, row in short_policies.sort_values("SOH_median").iterrows():
468         mark = "寿命最短" if row["shortest_strategy"] == "是" else "典型短寿命"
469         lines.append(
470             f"| `{row['policy']}` | {int(row['battery_count'])} | {row['SOH_median']:.6f} | "
471             f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** | "
472         )
473         lines.extend(
474             [
475                 "",
476                 "## 解释限制",
477                 "",
478                 "- 各策略的完整 200 循环样本数较少，结论用于本数据集内的典型策略比较，不宜直接外推为因果结论。",
479                 "- `4_8C_80PER_4_8C_NEWSTRUCTURE` 的中位数较高但包含一个明显较低的 SOH 个体，需结合离散程度解读。",
480                 "- 数据未覆盖 SOH<80% 的真实寿命终止循环，因此这里比较的是第 200 次循环的健康保持程度。",
481             ]
482         )
483     doc_path.parent.mkdir(parents=True, exist_ok=True)
484     doc_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
485
486 def write_policy_report(
487     report_path: Path,
488     policy_summary: pd.DataFrame,
489     critical_soh_low: float,

```



```

490     critical_soh_high: float,
491 ) -> None:
492     """输出第三次迭代要求的统一策略统计报告。"""
493     lines = [
494         "# 统一充电策略统计报告",
495         "",
496         "## 统计口径",
497         "",
498         "- 策略名称已按第三次迭代规则合并，并移除 `_NEWSTRUCTURE` 后缀。",
499         "- 最终 SOH 为完整记录至第 200 次循环的电池在 `cycle=200` 时的清洗后 SOH。"
500         ,
501         "- 充电时间、内阻和温度先按电池对 1-200 次循环取均值，再按策略汇总。",
502         f"- 下四分位线 Q1={critical_soh_low:.6f}; 上四分位线 Q3={critical_soh_high:.6f}。",
503         "",
504         "## 最终 SOH 与充电时间",
505         "",
506         "| 统一策略 | 电池记号 | 电池数 | 最终SOH平均 | 最小 | 最大 | 充电时间平均/"
507         "min | 最小/min | 最大/min |",
508         "|---|---|---:|---:|---:|---:|---:|---:|",
509     ]
510     for _, row in policy_summary.iterrows():
511         lines.append(
512             f"| `{row['policy']}` | {row['battery_ids']} | {int(row['battery_count'])} | "
513             f"{row['SOH_mean']:.6f} | {row['SOH_min']:.6f} | {row['SOH_max']:.6f} | "
514             f"{row['charge_time_mean']:.6f} | {row['charge_time_min']:.6f} | {row['charge_time_max']:.6f} | "
515         )
516     lines.extend(
517         [
518             "",
519             "## 其他参数",
520             "",
521             "| 统一策略 | C1取值 | Q1取值 | C2取值 | 初始容量平均/Ah | 最小/Ah | 最大/Ah | 平均内阻 | 平均温度/°C |",
522             "|---|---|---|---:|---:|---:|---:|---:|",
523         ]
524     )
525     for _, row in policy_summary.iterrows():
526         lines.append(
527             f"| `{row['policy']}` | {row.get('C1_values', '')} | {row.get('Q1_values', '')} | "

```

```

526         f"{row.get('C2_values', '')} | {row['initial_capacity_mean']:.6f} | "
527         f"{row['initial_capacity_min']:.6f} | {row['initial_capacity_max']:.6f} | "
528         f"{row['IR_mean']:.6f} | {row['Tavg_mean']:.3f} | "
529     )
530     lines.extend(
531         [
532             "",
533             "## 解释限制",
534             "",
535             "- 9块仅有150次循环数据的测试电池未纳入最终SOH策略统计。",
536             "- 统一策略是按题目指定规则进行的分析分组；报告中的相关性不代表充电策略对寿命的因果效应。",
537             "- 本数据片段尚未覆盖 SOH<80% 的真实寿命终止点，最终SOH用于比较第200次循环时的健康保持程度。",
538         ]
539     )
540     report_path.parent.mkdir(parents=True, exist_ok=True)
541     report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
542
543
544 def build_acceptance_results(
545     original_summary: pd.DataFrame,
546     normalized_summary: pd.DataFrame,
547     normalized_cycles: pd.DataFrame,
548     policy_table: pd.DataFrame,
549     cycle_200: pd.DataFrame,
550     policy_summary: pd.DataFrame,
551     soh_dir: Path,
552     table_write_paths: dict[str, Path],
553 ) -> list[tuple[str, bool, str]]:
554     """执行第三次迭代的显式验收检查。"""
555     original_count = int(original_summary["policy"].nunique())
556     normalized_count = int(normalized_summary["policy"].nunique())
557     expected_names = {normalize_policy_name(value) for value in original_summary["policy"].unique()}
558     actual_names = set(normalized_summary["policy"].unique())
559     forbidden = {"8OPER_3_6C", "4_8C_8OPER_4_8C_NEWSTRUCTURE"}
560     plot_files = list(soh_dir.glob("SOH_*.png"))
561     individual_files = [path for path in plot_files if path.stem != "SOH_ALL"]
562     results = [
563         (
564             "策略数量恰好减少2种",
565             original_count - normalized_count == 2,

```

```

566         f"原始{original_count}种，统一后{normalized_count}种，减少{
           original_count - normalized_count}种",
567     ),
568     (
569         "统一策略集合正确",
570         actual_names == expected_names,
571         f"实际策略: {sorted(actual_names)}",
572     ),
573     (
574         "旧策略名称已消除",
575         not (actual_names & forbidden),
576         f"禁止名称残留: {sorted(actual_names & forbidden)}",
577     ),
578     (
579         "NEWSTRUCTURE后缀已全部移除",
580         not any("NEWSTRUCTURE" in name.upper() for name in actual_names),
581         "统一后的策略名不含NEWSTRUCTURE",
582     ),
583     (
584         "全部处理表命名一致",
585         set(normalized_cycles["policy"].unique()) == actual_names
586         and set(policy_table["policy"].unique()) == actual_names
587         and set(cycle_200["policy"].unique()).issubset(actual_names),
588         "清洗循环表、charge_policy和battery_SOH_charge均采用统一名称",
589     ),
590     (
591         "charge_policy覆盖49块电池",
592         len(policy_table) == 49
593         and policy_table["battery_id"].nunique() == 49,
594         f"记录数={len(policy_table)}, 唯一电池数={policy_table['battery_id'].
           nunique()}",
595     ),
596     (
597         "核心CSV均写入标准文件名",
598         all(path.name == expected for expected, path in table_write_paths.
           items()),
599         ", ".join(f"{expected}->{path.name}" for expected, path in
           table_write_paths.items()),
600     ),
601     (
602         "策略报告覆盖全部统一策略",
603         set(policy_summary["policy"]) == actual_names and len(policy_summary)
           == normalized_count,
604         f"报告策略数={len(policy_summary)}",

```

```

605     ),
606     (
607         "49张单体曲线与SOH_ALL齐全",
608         len(individual_files) == 49 and (soh_dir / "SOH_ALL.png").is_file(),
609         f"单体曲线={len(individual_files)}, SOH_ALL={int((soh_dir / 'SOH_ALL.
        png').is_file())}",
610     ),
611 ]
612 return results
613
614
615 def write_acceptance_report(
616     path: Path,
617     results: list[tuple[str, bool, str]],
618     mapping: dict[str, str],
619 ) -> None:
620     lines = [
621         "# 第三次迭代验收确认",
622         "",
623         "| 验收项 | 结果 | 证据 |",
624         "|---|---|---|",
625     ]
626     for name, passed, evidence in results:
627         lines.append(f"| {name} | {'通过' if passed else '失败'} | {evidence} |")
628     lines.extend(["", "## 策略名称映射", "", "| 原始名称 | 统一名称 |", "
        |---|---|"])
629     for source, target in mapping.items():
630         lines.append(f"| `{source}` | `{target}` |")
631     lines.extend(["", f"**总体结论: {'全部通过' if all(item[1] for item in
        results) else '存在失败项'}.**"])
632     path.parent.mkdir(parents=True, exist_ok=True)
633     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
634
635
636 def write_csv_if_changed(data: pd.DataFrame, path: Path) -> Path:
637     """内容不变时避免重写，兼容 CSV 正被表格软件打开的情形。"""
638     if path.exists():
639         try:
640             existing = pd.read_csv(path)
641             pd.testing.assert_frame_equal(existing, data, check_dtype=False)
642             return path
643         except (AssertionError, OSError, ValueError):
644             pass
645     pending_path = path.with_name(f"{path.stem}_v3_pending{path.suffix}")

```

```

646     try:
647         data.to_csv(path, index=False, encoding="utf-8-sig")
648         if pending_path.exists():
649             pending_path.unlink()
650         return path
651     except PermissionError:
652         data.to_csv(pending_path, index=False, encoding="utf-8-sig")
653         return pending_path
654
655
656 def write_log(
657     log_path: Path,
658     summary: pd.DataFrame,
659     cycles: pd.DataFrame,
660     audit: list[dict[str, float | int | str]],
661     cycle_200: pd.DataFrame,
662     policy_summary: pd.DataFrame,
663     critical_soh_low: float,
664     critical_soh_high: float,
665     iqr_factor: float,
666     font_name: str,
667     output_paths: list[Path],
668     prompt_text: str,
669     policy_mapping: dict[str, str],
670     acceptance_results: list[tuple[str, bool, str]],
671 ) -> None:
672     counts = Counter(str(item["column"]) for item in audit)
673     max_cycle_counts = cycles.groupby("battery_id")["cycle"].max().value_counts
674         ().sort_index()
675     missing_summary = int(summary.isna().sum().sum())
676     missing_cycles = int(cycles.isna().sum().sum())
677     long_count = int((cycle_200["SOH"] >= critical_soh_high).sum())
678     short_count = int((cycle_200["SOH"] <= critical_soh_low).sum())
679     middle_count = len(cycle_200) - long_count - short_count
680     longest = policy_summary.loc[policy_summary["longest_strategy"] == "是", "
        policy"].iloc[0]
681     shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是", "
        policy"].iloc[0]
682     normalized_policy_count = len(set(policy_mapping.values()))
683
684     lines = [
685         "# 问题一第三次迭代运行日志 (log_3) ",
686         "",
687         f"- 运行时间: {datetime.now().astimezone().isoformat(timespec='seconds')}

```

```

        ",
687 f"- IQR 系数: {iqr_factor:.6g}",
688 f"- 上四分位线 Q3 (75%分位数): {critical_soh_high:.6f}",
689 f"- 下四分位线 Q1 (25%分位数): {critical_soh_low:.6f}",
690 f"- 绘图字体: {font_name}",
691 "",
692 "## 输入质量检查",
693 "",
694 f"- battery_summary.csv: {len(summary)} 行, {summary.shape[1]} 列, {
        summary['battery_id'].nunique()} 块电池。",
695 f"- cycle_train.csv: {len(cycles)} 行, {cycles.shape[1]} 列。",
696 f"- battery_id + cycle 重复记录: {int(cycles.duplicated(['battery_id', '
        cycle']).sum())}。",
697 f"- 汇总表缺失单元格: {missing_summary} (其中 C1 的结构性缺失保留, 不参与循
        环数据清洗)。",
698 f"- 循环表缺失单元格: {missing_cycles}。",
699 f"- 最大循环数分布: {dict((int(k), int(v)) for k, v in max_cycle_counts.
        items())}。",
700 "",
701 "## 策略统一命名",
702 "",
703 f"- 原始策略数: {len(policy_mapping)}; 统一后策略数: {
        normalized_policy_count}; 恰好减少 {len(policy_mapping) -
        normalized_policy_count} 种。",
704 "- `8OPER_3_6C` 统一为 `3_6C-8OPER_3_6C`。",
705 "- `4_8C_8OPER_4_8C_NEWSTRUCTURE` 统一为 `4_8C_8OPER_4_8C`。",
706 "- 其余策略仅移除 `_NEWSTRUCTURE` 后缀。",
707 "",
708 "## IQR 清洗",
709 "",
710 "- 方法: 按 battery_id 分组, 对 capacity、SOH、SOH_smooth、chargetime、IR
        、Tavg 分别计算 Q1、Q3 和 IQR。",
711 "- 判定: 小于 Q1 - 1.5×IQR 或大于 Q3 + 1.5×IQR 的值标记为异常。",
712 "- 填补: 使用同一电池、同一字段中距离最近的前后正常值均值; 边界点仅有一侧正
        常值时使用该侧值。",
713 f"- 共替换 {len(audit)} 个单元格; 分字段数量: {dict(counts)}。",
714 "- 原始 data 文件未改写; 清洗结果另存为 output/A/cycle_train_cleaned.csv。
        ",
715 "",
716 "## 200 次循环样本",
717 "",
718 f"- 纳入: 最大 cycle 恰为 200 的 {len(cycle_200)} 块电池。",
719 f"- 排除: 仅记录至 150 次循环的 {summary['prediction_test'].sum()} 块测试
        电池。",

```

```

720 f"- SOH 范围: {cycle_200['SOH'].min():.6f} - {cycle_200['SOH'].max():.6f}
    , 均值 {cycle_200['SOH'].mean():.6f}。",
721 f"- 按两个分位数分类: 长寿命 {long_count} 块, 中等寿命 {middle_count} 块,
    短寿命 {short_count} 块。",
722 f"- 寿命最长策略 (按策略 SOH 中位数): {longest}。",
723 f"- 寿命最短策略 (按策略 SOH 中位数): {shortest}。",
724 "",
725 "## SOH 曲线",
726 "",
727 "- 使用清洗后的 `SOH_smooth` 绘制49张单体曲线和1张全部电池曲线。",
728 "- 单体曲线命名为 `SOH_N.png`; 总图命名为 `SOH_ALL.png`, 颜色按电池编号由蓝
    到红渐变。",
729 "",
730 "## 验收确认",
731 "",
732 ]
733 for name, passed, evidence in acceptance_results:
734     lines.append(f"- [{ 'x' if passed else ' '}] {name}: {evidence}。")
735 lines.extend(["", "## 输出文件", ""])
736 lines.extend(f"- `{path.relative_to(PROJECT_ROOT).as_posix()}`" for path in
    output_paths)
737 lines.extend(
738     [
739         "",
740         "## 说明",
741         "",
742         "- 箱型图纵轴采用“第 200 次循环 SOH”作为使用寿命代理指标; 本题数据尚未
            覆盖 SOH<80% 的真实寿命终止循环。",
743         "- IQR 是统计异常筛查, 不等同于确认传感器故障; 清洗结果用于完成本题指定
            分析。",
744         "- 图中的两条参考线统一命名为上四分位线 (Q3) 和下四分位线 (Q1)。",
745         "- 策略报告使用统一后的7种策略, 最终SOH统计只纳入完整200循环的40块电池。
            ",
746         "",
747         "## 原始提示词",
748         "",
749         "以下完整保留本次执行时 `A/问题一/AGENTS.md` 的内容: ",
750         "",
751         "``text",
752         prompt_text.rstrip(),
753         "``",
754     ]
755 )
756 log_path.parent.mkdir(parents=True, exist_ok=True)

```

```

757     log_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
758
759
760 def parse_args() -> argparse.Namespace:
761     parser = argparse.ArgumentParser(description="运行问题一完整分析流程")
762     parser.add_argument("--iqr-factor", type=float, default=1.5, help="IQR 异常
763         判定系数, 默认 1.5")
764     return parser.parse_args()
765
766 def main() -> None:
767     args = parse_args()
768     if args.iqr_factor <= 0:
769         raise ValueError("--iqr-factor 必须大于 0")
770
771     data_dir = PROJECT_ROOT / "data"
772     output_dir = PROJECT_ROOT / "output" / "A"
773     output_dir.mkdir(parents=True, exist_ok=True)
774     original_summary = pd.read_csv(data_dir / "battery_summary.csv")
775     original_cycles = pd.read_csv(data_dir / "cycle_train.csv")
776     validate_inputs(original_summary, original_cycles)
777     summary, cycles, policy_mapping = normalize_policies(original_summary,
778         original_cycles)
779     validate_inputs(summary, cycles)
780
781     cleaned, audit = iqr_clean_by_battery(cycles, MEASUREMENT_COLUMNS, args.
782         iqr_factor)
783     policy_table = build_policy_table(summary)
784     cycle_200 = build_cycle_200_summary(cleaned, policy_table)
785     critical_soh_low = float(cycle_200["SOH"].quantile(0.25))
786     critical_soh_high = float(cycle_200["SOH"].quantile(0.75))
787     policy_order = list(dict.fromkeys(summary.sort_values("battery_id")["policy"]
788         ))
789     policy_summary = build_policy_summary(
790         cycle_200,
791         policy_order,
792         critical_soh_low,
793         critical_soh_high,
794         summary,
795         cleaned,
796     )
797
798     cleaned_path = output_dir / "cycle_train_cleaned.csv"
799     policy_path = output_dir / "charge_policy.csv"

```



```

797 summary_path = output_dir / "battery_SOH_charge.csv"
798 soh_example_path = output_dir / "SOH_1_example.png"
799 histogram_path = output_dir / "SOH_summary.png"
800 scatter_path = output_dir / "SOH_chargetime.png"
801 boxplot_path = output_dir / "charge_policy.png"
802 policy_summary_path = output_dir / "policy.csv"
803 report_path = PROJECT_ROOT / "output" / "doc" / "1.md"
804 acceptance_path = output_dir / "acceptance_3.md"
805 soh_dir = PROJECT_ROOT / "A" / "问题一" / "SOH"
806
807 cleaned_write_path = write_csv_if_changed(cleaned, cleaned_path)
808 policy_write_path = write_csv_if_changed(policy_table, policy_path)
809 summary_write_path = write_csv_if_changed(cycle_200, summary_path)
810 policy_summary_write_path = write_csv_if_changed(policy_summary,
811         policy_summary_path)
812
813 font_name = configure_chinese_font()
814 plot_battery_soh(cleaned, 1, soh_example_path)
815 save_soh_histogram(cycle_200, histogram_path, critical_soh_low,
816         critical_soh_high)
817 save_soh_charge_scatter(cycle_200, scatter_path, policy_order)
818 save_policy_boxplot(
819     cycle_200,
820     boxplot_path,
821     critical_soh_low,
822     critical_soh_high,
823     policy_order,
824 )
825 write_policy_report(
826     report_path,
827     policy_summary,
828     critical_soh_low,
829     critical_soh_high,
830 )
831
832 soh_dir.mkdir(parents=True, exist_ok=True)
833 for stale_plot in soh_dir.glob("SOH_*.png"):
834     stale_plot.unlink()
835 for battery_id in sorted(int(value) for value in cleaned["battery_id"].
836     unique()):
837     plot_battery_soh(cleaned, battery_id, soh_dir / f"SOH_{battery_id}.png")
838 plot_all_battery_soh(cleaned, soh_dir / "SOH_ALL.png")
839
840 acceptance_results = build_acceptance_results(

```

```

838     original_summary,
839     summary,
840     cleaned,
841     policy_table,
842     cycle_200,
843     policy_summary,
844     soh_dir,
845     {
846         cleaned_path.name: cleaned_write_path,
847         policy_path.name: policy_write_path,
848         summary_path.name: summary_write_path,
849         policy_summary_path.name: policy_summary_write_path,
850     },
851 )
852 write_acceptance_report(acceptance_path, acceptance_results, policy_mapping)
853
854 output_paths = [
855     cleaned_write_path,
856     policy_write_path,
857     summary_write_path,
858     soh_example_path,
859     histogram_path,
860     scatter_path,
861     boxplot_path,
862     policy_summary_write_path,
863     report_path,
864     acceptance_path,
865     soh_dir,
866 ]
867 write_log(
868     PROJECT_ROOT / "A" / "问题一" / "log_3.md",
869     summary,
870     cycles,
871     audit,
872     cycle_200,
873     policy_summary,
874     critical_soh_low,
875     critical_soh_high,
876     args.iqr_factor,
877     font_name,
878     output_paths,
879     (PROJECT_ROOT / "A" / "问题一" / "AGENTS.md").read_text(encoding="utf-8")
880     ,
881     policy_mapping,

```

```

881         acceptance_results,
882     )
883
884     print(f"IQR 替换单元格数: {len(audit)}")
885     print(f"200 次循环样本数: {len(cycle_200)}")
886     print(f"critical_SOH_low: {critical_soh_low:.6f}")
887     print(f"critical_SOH_high: {critical_soh_high:.6f}")
888     print(f"策略数量: {len(policy_mapping)} -> {len(set(policy_mapping.values()))}")
889     print(f"SOH曲线数量: {len(list(soh_dir.glob('SOH_*.png')))}")
890     failed = [name for name, passed, _ in acceptance_results if not passed]
891     print(f"验收失败项: {failed}")
892     for path in output_paths:
893         print(f"已生成: {path}")
894
895
896 if __name__ == "__main__":
897     main()

```

C AI 工具使用声明

根据赛事论文格式规范，现就人工智能工具使用情况作如下声明。全文不出现参赛者姓名与学校信息。

C.1 所用工具

- OpenAI ChatGPT（记录版本：1.2026.190，模型 gpt5.6-sol-high）
- OpenAI Codex CLI（记录版本：v0.147.0，模型 gpt5.6-sol-high；DeepSeek-V4-Flash-0731）
- 对话式编程助手（用于文献整理、问题一结果写入 LaTeX 论文框架）

C.2 使用目的与环节

1. 讨论与协作：询问 Git 多人协作与仓库管理相关操作说明。
2. 问题一：依据人工撰写的 AGENTS.md 生成模块化分析代码，完成 IQR 清洗、策略提取、200 圈汇总与绘图；人工检查可运行性后调整参数并复现日志。
3. 文献：整理快充、SOH 预测与策略优化相关公开文献条目，写入 references/。
4. 论文半成品：将仓库已有问题一成果写入 LaTeX 正文，并为问题二至四保留章节框架、符号与拟采用方法。

C.3 提示方式

主要方式为：先由人工完成题意分析与方法构想，再将任务写成 AGENTS.md 或结构化提示，交由 AI 生成代码或文档草稿，随后人工运行、核验数值与图件是否与数据一致。问题一提示词原文见 A/问题一/AGENTS.md 及 log_2.md 附录，此处不重复粘贴以免附录过长；支撑材料中保留全文。

C.4 采纳、修改与核验

- 问题一流水线经运行日志确认：IQR 替换 559 格、200 圈样本 40 块、分位阈值 0.993433/0.996103，与正文一致。
- AI 生成的解释性文字（如 NEWSTRUCTURE 含义、机制讨论）均对照数据表核验后写入；涉及因果的表述已降级为初步观察。
- 问题二至四的模型尚未由 AI 或人工给出最终数值，正文相应部分明确标注为框架，不作为已验证结论。

AI 仅作为辅助工具，论文观点、模型选择与最终结论由参赛队负责。